UNIVERSIDAD DE ANTIOQUIA

Facultad de ingeniería
Ingeniería de telecomunicaciones

Ude@book

Asignatura
Informatifca ll

Profesor
Anibal Jose Guerra

Presentado por
Andrés Felipe Sierra
David Guardiola

October 19, 2026

1 Descripción del Proyecto

UdeABook es una red social en la que los usuarios podrán crear un perfil utilizando un nombre de usuario único. A través de este perfil, tendrán acceso a un menú principal desde el cual podrán agregar otros usuarios como amigos, visitar perfiles, recibir sugerencias de amistad, administrar mensajes y remitirlos a otros usuarios cuando sea necesario. La aplicación será desarrollada utilizando el framework *Qt* y el lenguaje de programación *C++*. Para su implementación se hará uso de clases que representarán tanto la aplicación principal como los diferentes componentes encargados de su estructura, funcionamiento e interacción entre los usuarios. De esta manera, se busca construir una plataforma organizada, modular y fácil de mantener, aplicando conceptos de programación orientada a objetos para facilitar su desarrollo y escalabilidad.

2 Análisis del Problema

El proyecto UdeABook se dividirá en diferentes responsabilidades, representadas por medio de clases que ayudarán a mantener el código más organizado, legible y fácil de mantener. Para esta entrega preliminar se han identificado varias clases que permitirán construir la estructura inicial del sistema. La clase **UdeABook** se encargará de gestionar la interacción con el usuario, permitiendo realizar acciones como registrarse, iniciar sesión y acceder a las diferentes funcionalidades de la aplicación. Por su parte, la clase **Usuario** almacenará la información de cada persona registrada, como su apodo, contraseña, país, ciudad, fecha de inscripción, amigos y mensajes enviados o recibidos, entre otros datos. También se tendrá la clase **Álbum**, que permitirá organizar las fotografías que cada usuario desee guardar y separar. Para esto, se implementará la clase **Foto**, encargada de almacenar la información de cada fotografía, como su ubicación en el sistema, fecha de captura y cantidad de "me gusta". Cada fotografía pertenecerá a un álbum. Por otra parte, se construirá la clase **Mensaje**, que almacenará la información de los mensajes enviados entre usuarios, incluyendo

su contenido, fecha de envío y estado de lectura. A medida que avance el desarrollo, se podrán realizar ajustes al diseño inicial e incorporar los atributos y métodos que sean necesarios para cumplir con las funcionalidades del proyecto.

3 Diagrama de clases

El siguiente diagrama de clases presenta la estructura del sistema, identificando las clases que lo componen, sus atributos, métodos y las relaciones establecidas entre ellas. Esta representación permite comprender la organización del diseño y la interacción entre sus componentes.

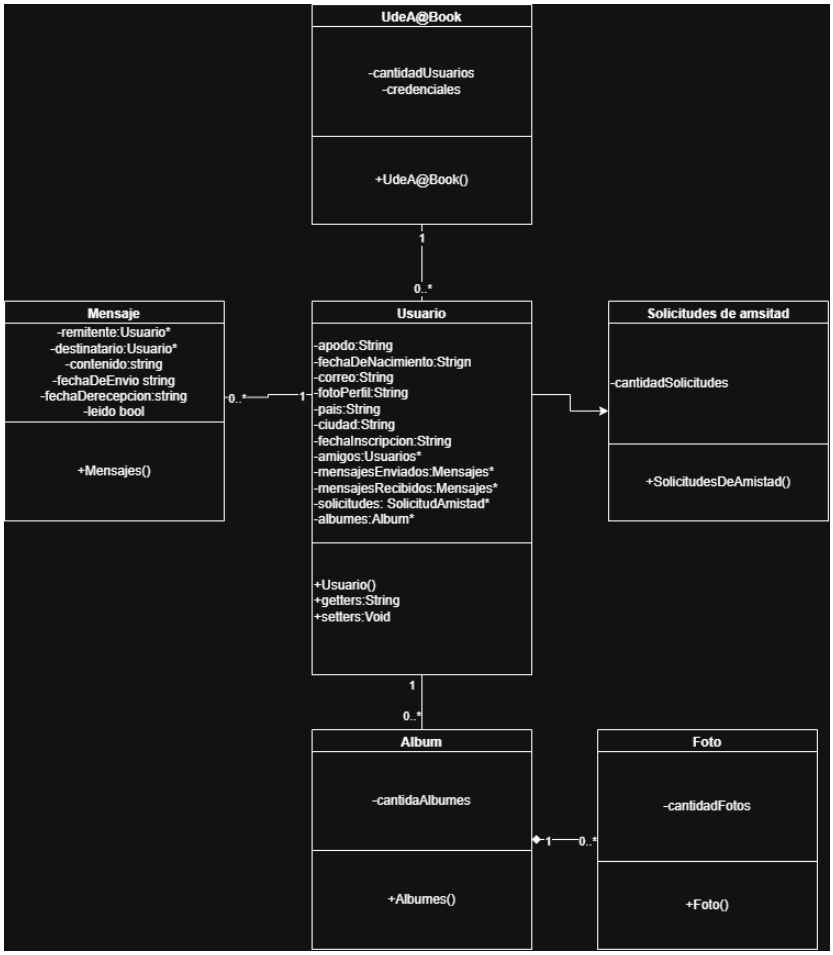


Figure 1: Diagrama de clases del sistema

“‘latex

4 Diseño e implementación de las clases

Para el desarrollo de UdeA@Book se ha definido una arquitectura orientada a objetos en C++, en la que cada clase representa un componente específico de la red social. Esta separación permite organizar la información, distribuir las responsabilidades y facilitar la implementación progresiva de las funcionalidades.

4.1 Clase Usuario

La clase `Usuario` representa a cada persona registrada en la aplicación. Contiene los atributos correspondientes a su información personal, como apodo, fecha de nacimiento, correo electrónico, fotografía de perfil, país, ciudad y fecha de inscripción.

Además, esta clase administra las colecciones asociadas al usuario, entre ellas los mensajes enviados y recibidos, las solicitudes de amistad y los álbumes. También representa sus relaciones de amistad mediante referencias a otros objetos de tipo `Usuario`, sin necesidad de crear una clase independiente para los amigos.

Durante la implementación se han incorporado constructores, métodos de consulta (*getters*) y modificación (*setters*). Asimismo, se han establecido las bases para administrar las colecciones mediante memoria dinámica.

4.2 Clase RedSocial

La clase `RedSocial` se encarga de coordinar el funcionamiento general de la red y mantener el registro de usuarios. Entre sus responsabilidades se encuentran la búsqueda y el registro de usuarios, así como la coordinación de las operaciones que involucran a varios usuarios.

Esta clase también será responsable de gestionar el envío y la aceptación de solicitudes de amistad, garantizando que las relaciones establecidas sean mutuas. De esta manera, se separa la administración general de la red de los datos y comportamientos propios de cada usuario.

4.3 Clases complementarias

Se implementaron las clases `Mensajes`, `Solicitud` y `Album`, cada una con responsabilidades específicas dentro de la aplicación UdeA@Book. La clase `Mensajes` representa la comunicación entre usuarios y almacena información sobre el remitente, el destinatario, el contenido, la fecha y el estado de lectura. Además, incorpora métodos de consulta y una operación para marcar un mensaje como leído. La clase `Solicitud` permite representar las solicitudes de amistad entre usuarios. Cada solicitud almacena referencias al remitente y al destinatario, así como un estado que permite identificar si está pendiente, fue aceptada o fue

rechazazada. Por su parte, la clase `Album` representa los álbumes asociados a un usuario. Contiene los atributos `nombre` y `ruta`, que permiten identificar el álbum y consultar la ubicación de sus fotografías. Estas clases se relacionan con `Usuario` mediante arreglos dinámicos de punteros, lo que permite organizar la información sin duplicar innecesariamente los objetos relacionados.

4.4 Organización del almacenamiento

Para administrar las colecciones se ha optado por utilizar arreglos dinámicos de punteros, acompañados de contadores de elementos y variables de capacidad. Esta estructura permite ampliar el espacio reservado cuando sea necesario, sin depender de contenedores de la biblioteca estándar de C++.

El diseño también distingue entre la memoria reservada para los arreglos y los objetos a los que apuntan sus elementos. Esta distinción permite establecer responsabilidades de administración de memoria y reducir el riesgo de fugas y liberaciones duplicadas.

La implementación se está desarrollando de forma incremental, incorporando y comprobando los componentes de manera progresiva antes de integrar las funcionalidades generales de la aplicación. “‘

5 Avances de implementación

Hasta el momento, se ha desarrollado la estructura inicial de las clases `Usuario`, `RedSocial`, `Mensajes`, `Solicitud`, `Album` y `MedidorRecursos`. El desarrollo se ha realizado de manera incremental, implementando y probando las funcionalidades antes de continuar con las siguientes. La clase `Usuario` almacena la información personal de cada persona registrada, incluyendo su apodo, fecha de nacimiento, correo electrónico, fotografía de perfil, país, ciudad y fecha de inscripción. También administra colecciones dinámicas relacionadas con las amistades, los mensajes enviados y recibidos, las solicitudes de amistad y los álbumes.

5.1 Gestión de amistades

Se implementaron operaciones para agregar, consultar y eliminar relaciones de amistad dentro de la colección de cada usuario. La operación para agregar un amigo verifica que la referencia recibida sea válida, que no corresponda al mismo usuario y que la relación no exista previamente. Las amistades se representan mediante referencias a objetos de tipo `Usuario`, sin crear una clase independiente para los amigos. La coordinación de las relaciones mutuas corresponde a `RedSocial`, que permite centralizar las operaciones que involucran a dos usuarios. Las pruebas realizadas en consola permitieron verificar la incorporación de un amigo y la consulta de una relación existente.

5.2 Gestión de mensajes

La clase `Mensajes` almacena la información de cada mensaje, incluyendo el remitente, el destinatario, el contenido, la fecha y su estado de lectura. Se implementó el envío de mensajes entre usuarios. Durante esta operación, el mensaje se incorpora a la colección de mensajes enviados del remitente y a la colección de mensajes recibidos del destinatario. Ambas colecciones almacenan referencias al mismo objeto `Mensajes`, evitando mantener dos copias independientes de su contenido. También se implementó la operación para marcar un mensaje como leído. Las pruebas en consola permitieron comprobar el envío, la consulta de la información y la modificación del estado de lectura. Para mantener la integridad de los datos, se contempló el manejo de errores durante la incorporación del mensaje a las colecciones de ambos usuarios.

5.3 Gestión de álbumes

Se implementó la clase `Album`, cuyos atributos `nombre` y `ruta` permiten identificar un álbum y consultar la ubicación de sus fotografías. Se incorporaron métodos de consulta que permiten acceder a estos datos sin modificar directamente los atributos privados. La clase `Usuario` administra una colección dinámica de punteros a objetos `Album`, junto con un contador de elementos y una variable de capacidad. Cuando el espacio reservado resulta insuficiente, el arreglo se amplía para permitir almacenar nuevos álbumes. Como prueba, se crearon los álbumes denominados *Vacaciones* y *Universidad*. Se verificó su asociación con un usuario y la consulta de sus nombres y rutas.

Se implementó la clase `Solicitud`, que representa una petición de amistad entre dos usuarios. Cada objeto contiene referencias al remitente y al destinatario, además de un estado que identifica la situación de la solicitud. La clase `RedSocial` incorpora operaciones para enviar, aceptar y rechazar solicitudes. Al enviar una solicitud, se comprueba que los usuarios sean válidos, que no sean la misma persona y que no exista una amistad previa. También se verifica que no haya una solicitud pendiente duplicada en la misma dirección. Cuando se acepta una solicitud válida, se incorporan los usuarios a sus respectivas colecciones de amistades, garantizando la reciprocidad de la relación. Si no es posible completar la operación, se contempla la reversión de la incorporación realizada previamente. El rechazo cambia el estado de la solicitud sin establecer la amistad. Las pruebas realizadas permitieron verificar el envío de una solicitud, su estado inicial y la aceptación, así como la comprobación de la relación de amistad resultante entre ambos usuarios.

6 Implementación de la clase Album

Se implementó la clase Album, encargada de representar los álbumes asociados a cada usuario dentro de la aplicación UdeA@Book. Esta clase contiene los atributos nombre y ruta, que permiten identificar el álbum y almacenar la ubicación de sus fotografías.

Para acceder a esta información se implementaron métodos de consulta (getters), los cuales devuelven referencias constantes a los atributos de tipo std::string. De esta manera, se permite consultar la información del álbum sin modificar directamente sus atributos privados.

La clase Usuario administra una colección dinámica de punteros a objetos Album, lo que permite asociar varios álbumes a un mismo usuario y ampliar la capacidad de almacenamiento cuando sea necesario. Asimismo, se implementaron métodos para agregar álbumes y consultar sus elementos.

Como prueba de funcionamiento, se crearon dos álbumes, correspondientes a Vacaciones y Universidad, y se verificó la asociación de estos con un usuario y la consulta de su información. Los resultados obtenidos permitieron comprobar el funcionamiento de los métodos implementados.

7 Implementación del medidor de recursos

Con el propósito de evaluar el comportamiento de las funcionalidades implementadas, se desarrolló la clase `MedidorRecursos`, encargada de registrar las iteraciones instrumentadas y llevar una contabilidad explícita de determinadas reservas y liberaciones de memoria dinámica. La clase dispone de operaciones para iniciar la medición de una funcionalidad, registrar iteraciones, contabilizar reservas de memoria, descontar liberaciones y mostrar los resultados en consola. Al iniciar una medición se reinicia el contador de iteraciones, mientras que el registro de memoria conserva las reservas que todavía permanecen contabilizadas. La instrumentación se incorporó en operaciones relacionadas con las amistades, los mensajes, las solicitudes y los álbumes. También se registran iteraciones en determinados ciclos que recorren o copian elementos de los arreglos dinámicos. En las pruebas se verificó que el medidor registrara las ampliaciones de las colecciones. Por ejemplo, al ampliar los arreglos de mensajes enviados y recibidos, se contabilizan las reservas de los nuevos espacios y se descuentan las correspondientes a los arreglos anteriores. Asimismo, se incorporaron registros de liberación durante la destrucción de las estructuras instrumentadas. En una ejecución de prueba, el reporte final mostró siete iteraciones y cero bytes de memoria registrada al terminar las operaciones instrumentadas. Este resultado indica que las reservas y liberaciones incluidas en esa ejecución quedaron equilibradas dentro del registro del medidor. Es importante aclarar que el mecanismo implementado constituye una estimación parcial basada en la instrumentación explícita del programa. No representa el consumo total de memoria del proceso, puesto que todavía no contabiliza todas las variables locales, los parámetros, las

asignaciones internas de `std::string` ni todas las estructuras utilizadas. Del mismo modo, el contador de iteraciones corresponde a los ciclos instrumentados y no necesariamente al total de operaciones ejecutadas por el programa. Por lo tanto, los resultados deben interpretarse de acuerdo con el alcance actual del medidor y ampliarse conforme se integren las demás funcionalidades.

8 Medición de recursos computacionales

Para analizar el comportamiento de las funcionalidades implementadas en UdeA@Book, se incorporó la clase `MedidorRecursos`, encargada de registrar las iteraciones realizadas durante la ejecución de una funcionalidad y estimar el consumo de memoria asociado. La medición se integra en los métodos de las clases del sistema, teniendo en cuenta tanto las operaciones principales como las auxiliares necesarias para completar cada proceso. De esta manera, se busca representar el trabajo computacional realizado y los recursos empleados por las estructuras de datos utilizadas. **Conteo de iteraciones.** Se registran las operaciones relevantes que se ejecutan durante cada funcionalidad, incluyendo los recorridos de arreglos dinámicos y las llamadas a métodos auxiliares. El conteo permite estudiar cómo aumenta el trabajo realizado cuando crece la cantidad de usuarios, amistades, mensajes u otros elementos gestionados por la aplicación. **Estimación de memoria.** Se registra una estimación de la memoria asociada a los objetos, las estructuras dinámicas y las asignaciones efectuadas durante la ejecución. Este cálculo debe distinguir entre memoria actual, memoria reservada por capacidad y memoria temporal utilizada por una operación. Cuando se emplean tamaños estimados, los resultados se interpretan como aproximaciones y no como mediciones exactas del consumo total del proceso. El propósito de esta instrumentación es complementar las pruebas funcionales con información cuantitativa que permita evaluar el comportamiento de los algoritmos y las estructuras utilizadas.

9 Gestión de amistades y sugerencias

La gestión de las relaciones entre usuarios se centraliza en la clase `RedSocial`. La clase `Usuario` mantiene las referencias a sus amigos mediante un arreglo dinámico de punteros a objetos `Usuario`, evitando la creación de una clase independiente para representar las amistades. Las solicitudes de amistad permiten gestionar el proceso previo al establecimiento de una relación. Cuando una solicitud es aceptada, la amistad debe quedar registrada en ambos usuarios para garantizar su carácter mutuo. Adicionalmente, se implementó el mecanismo de sugerencias de amistad, que identifica usuarios con amigos en común. El procedimiento recorre las relaciones existentes, cuenta las coincidencias y organiza los candidatos según la cantidad de amigos compartidos, priorizando

aquellos con mayor afinidad dentro de la red. También contempla restricciones para evitar sugerir al propio usuario, a sus amigos actuales o a candidatos que no cumplan las condiciones establecidas por el algoritmo. La cantidad de sugerencias mostradas se controla mediante el parámetro correspondiente. Esta funcionalidad permite ilustrar cómo el análisis de las relaciones entre usuarios puede generar recomendaciones dentro de una red social.

10 Pruebas funcionales y resultados

Durante el desarrollo se realizaron pruebas progresivas para comprobar el funcionamiento de las clases y la interacción entre sus estructuras de datos. Entre las pruebas realizadas se encuentran:

1. **Mensajes:** envío de mensajes entre usuarios, registro en las colecciones de mensajes enviados y recibidos, consulta de sus datos y cambio del estado de lectura.
2. **Álbumes:** creación y registro de álbumes asociados a un usuario, con almacenamiento del nombre y la ruta correspondiente.
3. **Amistades:** incorporación de relaciones entre usuarios y consulta de su existencia.
4. **Medición de recursos:** registro de iteraciones y memoria estimada durante la ejecución de las funcionalidades instrumentadas.
5. **Sugerencias de amistad:** identificación de candidatos a partir de amigos en común, filtrado de usuarios no elegibles y organización de los resultados.

Los resultados definitivos de cada prueba se documentarán a partir de las ejecuciones realizadas en el proyecto. Las pruebas de integración deben verificar que las funcionalidades trabajen correctamente en conjunto y que las estructuras dinámicas mantengan la consistencia de sus datos.

11 Conclusiones

El desarrollo de UdeA@Book permitió aplicar los principios de la programación orientada a objetos en la construcción de una aplicación de red social. La distribución de responsabilidades entre clases facilita la organización de los datos y la implementación progresiva de las funcionalidades. La clase `Usuario` concentra la información personal y las relaciones de cada persona registrada, mientras que `Mensajes` y `Album` representan elementos asociados a los usuarios. Por su parte, `RedSocial` coordina las operaciones generales, como la gestión de usuarios, las solicitudes de amistad y la generación de sugerencias. El uso de arreglos dinámicos permite administrar colecciones cuya cantidad de elementos

puede variar durante la ejecución. Esta decisión también hace necesario prestar atención a la administración de memoria, la capacidad reservada y la liberación adecuada de los recursos. La incorporación de `MedidorRecursos` permite complementar las pruebas funcionales con un análisis de las iteraciones y de la memoria estimada. Esta información contribuye a comprender el costo computacional de las operaciones y a identificar cómo puede variar según el tamaño de los datos procesados. Finalmente, el proyecto constituye una aplicación práctica de los conceptos de clases, objetos, encapsulamiento, relaciones entre objetos y estructuras dinámicas. La validación integral de las funcionalidades y la presentación de sus resultados permiten establecer el grado de cumplimiento de los objetivos propuestos.